

BUILDING AN ARTIFICIAL NEURAL NETWORK

Predict whether the customer will cancel the hotel reservation

Artificial neural networks and Deep Learning

Autores:

Juan García Santos juan.gsantos@alumnos.upm.es

Guillermo Alonso Rello guillermo.alonsor@alumnos.upm.es

Janire Amesti Soler janire.amesti@alumnos.upm.es

Daniel Henríquez Quesada daniel.henriquez.quesada@alumnos.upm.es

Fecha 15/12/2023

Índice

1	Introducción.....	4
2	Proceso de diseño.....	6
2.1	Arquitectura inicial	6
2.2	Arquitectura inicial con early stopping y aumentando épocas.....	8
2.3	Arquitectura inicial modificando batch size	9
2.4	Arquitectura aumentando neuronas o capas.....	11
2.5	Arquitectura modificando la función de activación	17
2.6	Arquitectura modificando el optimizador	19
2.7	Reducción varianza en modelo Momentum	21
2.8	Reducción varianza y reducción del sesgo en el modelo Momentum.....	22
2.9	Reducción varianza en modelo Adam	24
2.10	Reducción varianza y sesgo en modelo Adam.....	25
2.11	Últimos modelos para disminución de sesgo y varianza Adam	27
3	Resultados finales	30
4	Conclusiones	33

Índice de figuras

Figura 1: Esquema para diseñar una arquitectura con bajo sesgo y varianza.	6
Figura 2: Diseño inicial. Métrica y función de perdida en el conjunto de entrenamiento y en el conjunto de validación a lo largo de las épocas.	7
Figura 3: Comportamiento del modelo con el diseño inicial aumentando épocas con early stopping. .	8
Figura 4: Comportamiento red con diseño inicial con diferentes batch size.	10
Figura 5: Comportamiento de la red aumentando neuronas y aumentando capas.	11
Figura 6: Comportamiento de la red aumentando una capa y aumentando la cantidad de neuronas.	12
Figura 7: Comportamiento de las redes aumentando las capas.	14
Figura 8: Comportamiento arquitectura aumentando capas o aumentando neuronas.	15
Figura 9: Comportamiento arquitectura aumentando en consideración las capas y las neuronas.	16
Figura 10: Comportamiento redes con diferentes funciones de activación.	18
Figura 11: Comportamiento redes de los mejores resultados obtenidos para diferentes optimizadores.	20
Figura 12: Comportamiento de una red con mecanismos para reducir la varianza.	21
Figura 13: Comportamiento mejor modelo momentum con técnicas para reducir varianza y aumentar sesgo.	23
Figura 14: Comportamiento de las técnicas de reducción de varianza en Adam	25
Figura 15: Comportamiento de los modelos Adam para reducir sesgo y varianza.	26
Figura 16: Comportamiento diferentes arquitecturas para reducir sesgo y varianza con el optimizador Adam	29
Figura 17: Comportamiento modelo final	30
Figura 18: Matriz de confusión conjunto de testeo.	31

Índice de tablas

Tabla 1: Clasificación de las variables numéricas.	4
Tabla 2: Arquitectura inicial red neuronal.	7
Tabla 3: Métricas obtenidas con un diseño inicial.	7
Tabla 4: Arquitectura red neuronal inicial con aumento de neuronas y con early stopping.	8
Tabla 5: Métricas obtenidas con un diseño inicial aumentando capas y con early stopping	9
Tabla 6: Arquitectura de una red con diferentes tamaños de mini-batch.	9
Tabla 7: Métricas obtenidas para diferentes batch-size con el diseño inicial.	10
Tabla 8: Diseño de la red incrementando la complejidad aumentando neuronas o aumentando capas.	11
Tabla 9: Métricas obtenidas aumentando neuronas y aumentando capas.	12
Tabla 10: Arquitectura red añadiendo una capa más y aumentando neuronas.	12
Tabla 11: Métricas obtenidas aumentando neuronas y aumentando capas.	13
Tabla 12: Arquitectura de redes aumentando el número de capas.	13
Tabla 13: Métricas obtenidas aumentando el número de capas	14
Tabla 14: Arquitectura red aumentando neuronas y aumentando capas.	14
Tabla 15: Métricas obtenidas aumentando el número de capas y el número de neuronas.	15

Tabla 16: Arquitectura de red aumentando en consideración las capas y las neuronas.	16
Tabla 17: Métricas obtenidas aumentando el número de capas y el número de neuronas en consideración.	17
Tabla 18: Arquitectura red modificando las funciones de activación.	17
Tabla 19: Métricas de los resultados para las redes con distintas funciones de activación.	18
Tabla 20: Arquitectura de red de los mejores resultados obtenidos para diferentes optimizadores. .	19
Tabla 21: Métricas de los mejores resultados obtenidos para diferentes optimizadores.	20
Tabla 22: Métricas de una red con mecanismos para reducir la varianza.	22
Tabla 23: Diseños de redes para disminuir la varianza y aumentar el sesgo.	23
Tabla 24: Métricas mejor modelo momentum con técnicas para reducir varianza y aumentar sesgo.	24
Tabla 25: Métricas mejor modelo Adam con técnicas para reducir la varianza del modelo.	24
Tabla 26: Diseños de redes para disminuir la varianza y el sesgo en modelo Adam.	25
Tabla 27: Métricas mejor modelo Adam con técnicas para reducir la varianza y sesgo del modelo ...	27
Tabla 28: Métricas diferentes arquitecturas para reducir sesgo y varianza con el optimizador Adam.	28
Tabla 29: Métricas de los últimos modelos propuestos para reducir sesgo y varianza	29
Tabla 30: Métricas modelo final.	30
Tabla 31: Precision y recall obtenidos en el conjunto de testeo.	31

1 Introducción

El objetivo de este proyecto reside en seguir el proceso de construcción de una red neuronal artificial profunda para resolver un problema de clasificación. Se utilizará un conjunto de datos de reservas de hoteles con el fin de predecir si un cliente honorará o cancelará su reserva.

El conjunto de datos utilizado para dar solución al problema consta de información sobre reservas de hotel. Los datos son una combinación del perfil del cliente y de las características de la reserva. Las dimensiones del dataset original proporcionado tiene 36275 filas y 19 variables.

A los datos se le aplica un procesamiento previo a la modelización. En cuanto a las variables categóricas, se aplica un tratamiento y limpieza de datos para posibilitar la modelización. La columna *"type of meal plan"* tiene una categoría minoritaria con solo 5 ejemplos, por lo que se decide eliminar esas 5 filas. Las columnas *"room type reserved"* y *"market segment types"* tienen muchas categorías, y todas las minoritarias se agrupan en una sola. Las categorías de estas 3 columnas se pasan a valores enteros. Se eliminan 2 columnas irrelevantes y se pone aparte la variable objetivo *"booking status"*. La etiqueta 1 significa que no se cancela y la etiqueta 0 que sí, siendo la clase 1 la mayoritaria ya que es lo que ocurre en la vida real.

En cuanto al resto de columnas, se diferencian las variables relacionadas con los perfiles de los clientes y las características de las reservas.

Tabla 1: Clasificación de las variables numéricas.

Perfiles clientes	Características reservas
• No. Adults • No. Children • No. Weekend nights • No. Week nighs • Type of mean plan • Required car parking space • Room type reserved • Lead time • Arrival month • Market segment type	• Repeated guest • No. previous cancelations • No. previous bookings not canceled • Avg. price per room • No. special requests

Analizando la matriz de covarianza, las correlaciones son bajas en realidad, a destacar la correlación entre el *"lead time"* y el *"booking status"* de -0.43, lo que apoya la explicación de que cuanto antes se haga la reserva, más sube la probabilidad de anularla.

Para final con el procesamiento previo, se aplica la normalización min-max estableciendo los valores en el intervalo -1 y 1, lo cual es un aspecto beneficioso para las redes.

El sistema de aprendizaje automático puede ayudar al hotel a prepararse para actuar en el caso de una probable cancelación de reserva. Si la red predice que el cliente va a cancelar, sería bastante

dañino equivocarse ya que podría haber un problema con ofrecer esa habitación a un cliente. Por otro lado, si se predice que el cliente no va a cancelar y luego cancela, es también dañino ya que el hotel podría haber dado esa habitación a otra persona y obtener más beneficios. Ambas cosas son problemáticas y determinar cuál lo es más depende de lo que vaya a hacer el hotel con la información. Por ejemplo, si el hotel confía en la red y pasa el primer caso, sería un problema más serio que el segundo, porque podría darles la misma habitación a dos personas distintas, lo cual es más serio que perder beneficios por mantener una habitación vacía. Como desconocemos la estrategia del hotel vamos a considerar una función de pérdida estándar que no se centre en disminuir ninguna parte específica de la matriz de confusión.

2 Proceso de diseño

En el apartado actual se analiza el proceso llevado a cabo para obtener los resultados finales, mostrando las arquitecturas de red intermedias implementadas junto con los hiperparámetros utilizados. En base al análisis del comportamiento de las arquitecturas, se toman decisiones y se justifican las futuras arquitecturas diseñadas. El objetivo consiste en encontrar el diseño de la red que reduzca el sesgo y la varianza obtenida. Por ello, las decisiones van a hacer continuamente referencia a la figura 1, donde se analiza el esquema a seguir para alcanzar el objetivo mencionado.

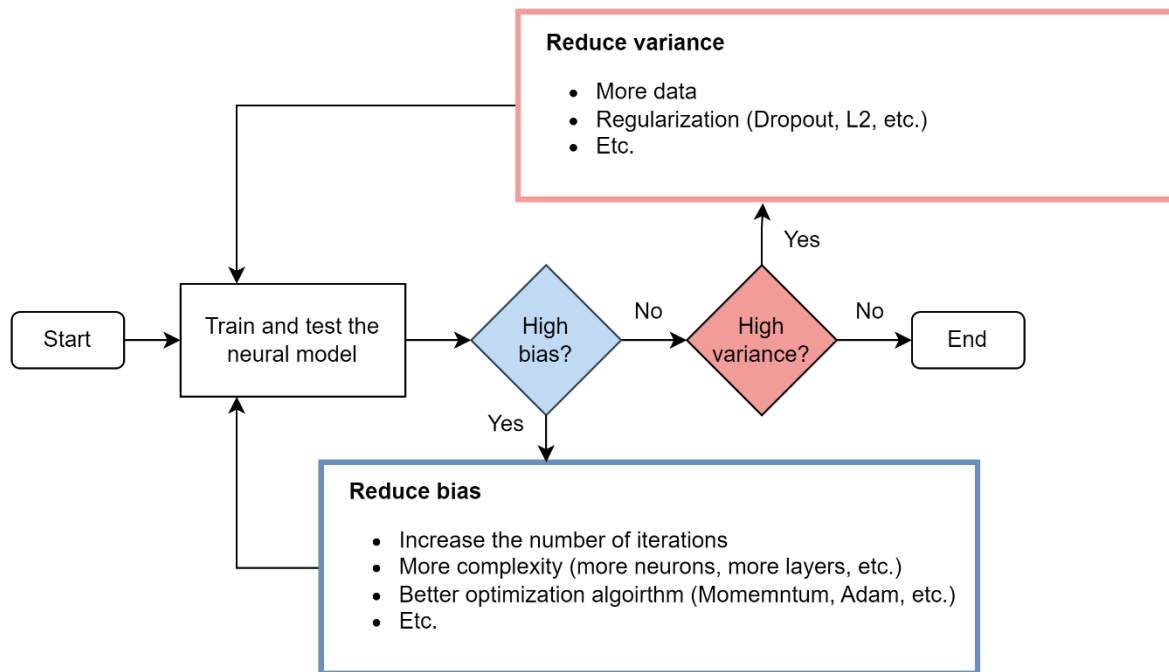


Figura 1: Esquema para diseñar una arquitectura con bajo sesgo y varianza.

El entrenamiento de las redes neuronales se realiza con el 80% de las instancias, se evalúa con el 10% de las instancias, y se mantiene un 10% de los datos para testearlo finalmente con el modelo final.

En cuanto al diseño, a nivel general, todas las arquitecturas implementadas cuentan con una capa de salida con una función de activación sigmoide que devuelve las probabilidades de pertenecer a cada una de las clases, ya que se trata de un problema de clasificación binaria. Asimismo, se utiliza la función de pérdida “*binary cross entropy*” debido a que se trata de un problema de clasificación binaria. Esta función mide la discrepancia entre la distribución de probabilidad predicha por el modelo y la distribución de probabilidad real de los datos de entrenamiento, por lo que el entrenamiento se centrará en optimizar dicho valor. Por otro lado, la métrica analizada es “*binary accuracy*” que evalúa la proporción de predicciones correctas realizadas por el modelo con respecto al total de ejemplos.

2.1 Arquitectura inicial

La estructura inicial con la que parte el proceso mostrado en la tabla 2 consiste en una red con una única capa oculta de 64 neuronas y con una función de activación *relu*. El learning rate inicial se establece en 0.01 y se ejecuta el algoritmo durante 100 épocas. Asimismo, en cada época se realiza una única iteración utilizando todos los datos de entrenamiento (batch) al conjunto.

Tabla 2: Arquitectura inicial red neuronal.

Neuronas capas ocultas	Hiperparámetros
64 neuronas	• Función activación: relu • Optimizador: SGD • Learning rate: 0.01 • Épocas: 200 • Batch

- Análisis del comportamiento

Analizando los resultados obtenidos en la tabla 3 y el comportamiento del modelo a lo largo de las iteraciones ilustrado en la figura 2, se observa que el sesgo es relativamente bajo, la diferencia entre el accuracy binario entre el conjunto de entrenamiento y validación es inferior al 1%. Por el contrario, analizando el sesgo del modelo, se percibe un sesgo alto teniendo en cuenta que el error humano es de un 5%. El comportamiento humano es 11% mejor al resultado del modelo.

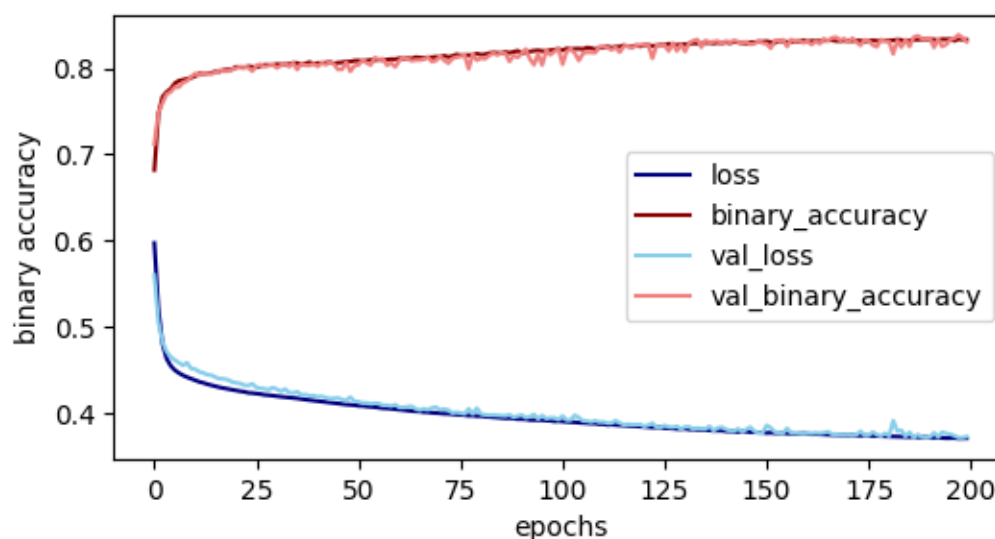


Figura 2: Diseño inicial. Métrica y función de pérdida en el conjunto de entrenamiento y en el conjunto de validación a lo largo de las épocas.

Tabla 3: Métricas obtenidas con un diseño inicial.

Binary accuracy entrenamiento	0.834
Binary accuracy validación	0.839
Diferencia binary accuracy entrenamiento y validación	0.48%
Diferencia binary accuracy train y error humano	11.59%
Coste computacional por época (segundos)	2.82

En consecuencia, se procede a mejorar el sesgo del modelo. No obstante, antes de comenzar con las técnicas que nos permiten reducir el sesgo, se ha observado un comportamiento que se estabiliza en

las épocas finales, por lo que se ha procedido a establecer un *early stopping*, para evitar entrenar épocas innecesarias por la falta de mejora en ambos conjuntos. Por lo tanto, el siguiente modelo sigue la misma estructura estableciendo mayor cantidad de épocas y un *early stopping* con una paciencia de 20 épocas y enfocándose en el binary accuracy del conjunto de validación.

2.2 Arquitectura inicial con early stopping y aumentando épocas

El diseño completo de la red añadiendo las características comentadas se muestra en la tabla 4.

Tabla 4: Arquitectura red neuronal inicial con aumento de neuronas y con early stopping.

Neuronas capas ocultas	Hiperparámetros
64 neuronas	• Función activación: relu • Optimizador: SGD • Learning rate: 0.01 • Épocas: 250 • Batch • Early-stopping (patience 20)

- Análisis del comportamiento

La figura 3 ilustra la necesidad de añadir el mecanismo de parada, ya que sobre la época 145 el entrenamiento se detiene por falta de mejora. La ejecución del entrenamiento al completo con las 250 épocas implicaría un aumento en el coste computacional sin generar mejora alguna en los resultados.

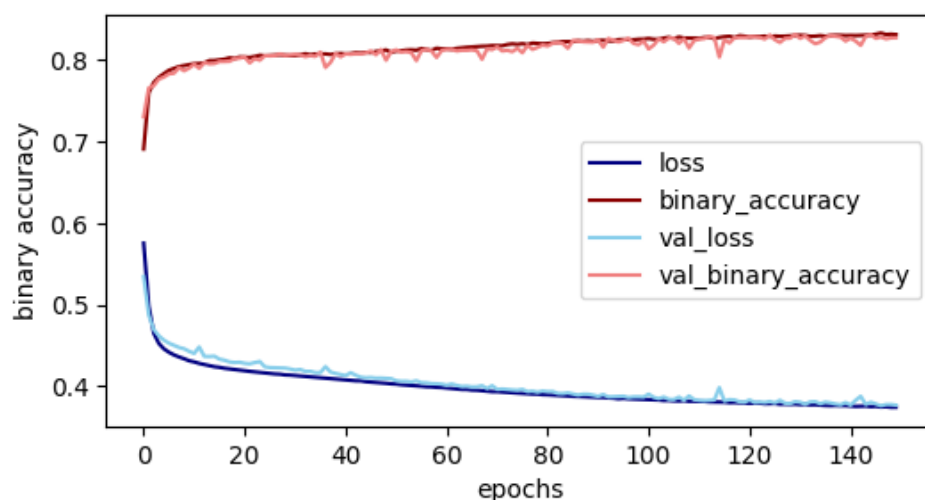


Figura 3: Comportamiento del modelo con el diseño inicial aumentando épocas con early stopping.

Analizando los resultados números finales obtenidos, los resultados no han variado. La varianza sigue siendo mínima y el sesgo teniendo en cuenta la diferencia con respecto al error humano sigue siendo alta, del 11%.

Tabla 5: Métricas obtenidas con un diseño inicial aumentando capas y con early stopping

Binary accuracy entrenamiento	0.831
Binary accuracy validación	0.830
Diferencia binary accuracy entrenamiento y validación	0.12%
Diferencia binary accuracy train y error humano	11.99%
Coste computacional por época (segundos)	3.75

A continuación, con el fin de intentar mejorar el sesgo del modelo, se va a analizar el comportamiento de un entrenamiento con *mini batches*, empleando un subconjunto de datos en cada iteración del entrenamiento. Una época corresponderá, en consecuencia, a un conjunto de iteraciones hasta entrenar todas las instancias del conjunto de entrenamiento.

2.3 Arquitectura inicial modificando batch size

Teóricamente, este enfoque es más exploratorio, permitiendo escapar de los óptimos locales, y, asimismo, el entrenamiento es más rápido, ya que los tamaños del *mini batch* implementados encajan en la memoria del GPU; 128, 256 y 512. El resto del diseño de la red permanece tal como en el modelo anterior, con las características mostradas en la tabla 6.

Tabla 6: Arquitectura de una red con diferentes tamaños de mini-batch

Neuronas capas ocultas	Hiperparámetros
64 neuronas	• Función activación: relu • Optimizador: SGD • Learning rate: 0.01 • Épocas: 250 • Mini-batch: 128, 256, 512 • Early-stopping (patience 20)

- Análisis del comportamiento

Analizando el comportamiento para diferentes valores de tamaño de *batch* ilustrados en la figura 4, visualmente no se aprecia diferencia significativa en los 3 entrenamientos, ni tampoco con el comportamiento analizado en el entrenamiento sin establecer *mini-batches*.

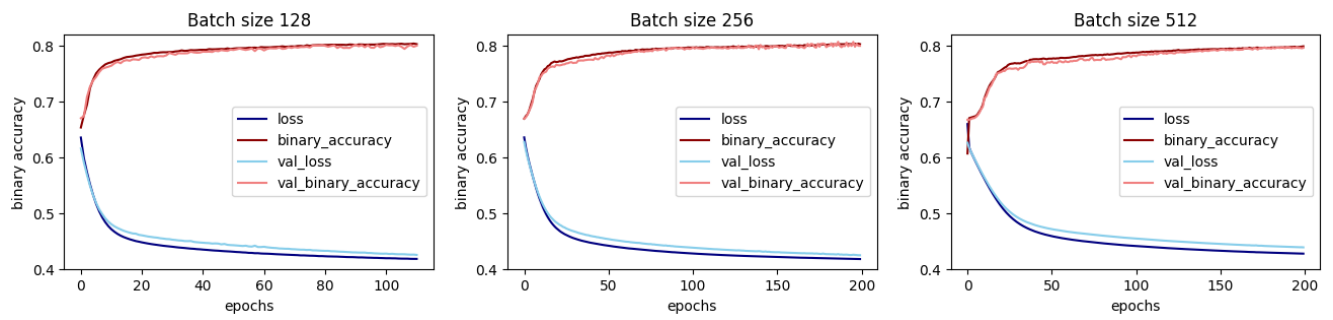


Figura 4: Comportamiento red con diseño inicial con diferentes batch size.

Los resultados numéricos de cada entrenamiento mostrados en la tabla 7 permiten obtener mayores conclusiones con respecto a la calidad de los modelos.

Tabla 7: Métricas obtenidas para diferentes batch-size con el diseño inicial.

Batch-size	128	256	512
Binary accuracy entrenamiento	0.801	0.802	0.797
Binary accuracy validación	0.801	0.806	0.797
Diferencia binary accuracy entrenamiento y validación	0.08%	0.45%	0.021%
Diferencia binary accuracy train y error humano	14.93%	14.81%	15.34%
Coste computacional por época (segundos)	0.712	0.429	0.412

Analizando los resultados, se concluye que las métricas de la bondad del ajuste son aproximadamente un 3% inferiores a los entrenamientos previos, por lo que establecer un tamaño *batch* para esta estructura de datos no ha beneficiado el entrenamiento. El sesgo también ha aumentado. No obstante, el tiempo requerido en segundos para entrenar cada época se ha reducido en un 81%, 88% y en un 89% con respecto al entrenamiento sin utilizar *mini-batch*.

Teniendo en cuenta que el entrenamiento se realiza con una arquitectura y un diseño simple, con una única capa, y que el objetivo va a consistir en reducir el sesgo que se está analizando en los resultados, la arquitectura se irá acomplejando, obteniendo resultados mejores y de menor sesgo con respecto al error humano. En consecuencia, es beneficioso continuar con una arquitectura que conlleve un menor coste computacional, ya que puede permitir incrementar aún más la complejidad de la red alcanzando resultados mejores tanto en la calidad de los resultados, es decir, en el sesgo, como en el coste computacional que suponen.

Por consiguiente, los entrenamientos siguientes consisten en aumentar la complejidad de la red con el objetivo de reducir el sesgo analizado, manteniendo un entrenamiento utilizando *mini-batch* dadas las ventajas que proporciona. En concreto los entrenamientos se realizan con un tamaño de *batch* de 128 o 256 dados los resultados obtenidos.

El siguiente paso consta en comparar los resultados aumentando el número de neuronas de la capa o aumentando el número de capas de la red, y analizar si este camino ayuda a reducir el sesgo.

2.4 Arquitectura aumentando neuronas o capas

Las modificaciones consisten en aumentar el número de neuronas a 128 por un lado, y, por otro lado, añadir una nueva capa de 64 neuronas, obteniendo una red con dos capas de 64 neuronas.

Tabla 8: Diseño de la red incrementando la complejidad aumentando neuronas o aumentando capas.

Neuronas capas ocultas	Hiperparámetros	Neuronas capas ocultas	Hiperparámetros
128 neuronas	• Función activación: relu	64 neuronas 64 neuronas	• Función activación: relu
	• Optimizador: SGD		• Optimizador: SGD
	• Learning rate: 0.01		• Learning rate: 0.01
	• Épocas: 200		• Épocas: 200
	• Mini-batch: 128		• Mini-batch: 128
	• Early-stopping (patience 20)		• Early-stopping (patience 20)

- Análisis del comportamiento

El análisis de la figura 5 permite concluir las diferencias entre aumentar la cantidad de neuronas y aumentar el número de capas. Manteniendo en ambos gráficos el eje vertical en la misma escala, se muestra una mejora significativa en la pérdida como en la métrica aumentando el número de capas, manteniendo, además, el equilibrio entre el conjunto de entrenamiento y de validación sin causar un sobreajuste en los datos de entrenamiento. Es cierto que son necesarias más épocas para alcanzar el resultado, pero se debe a que aumentar las neuronas ha causado un estancamiento de la red sin capacidad de mejorar el resultado.

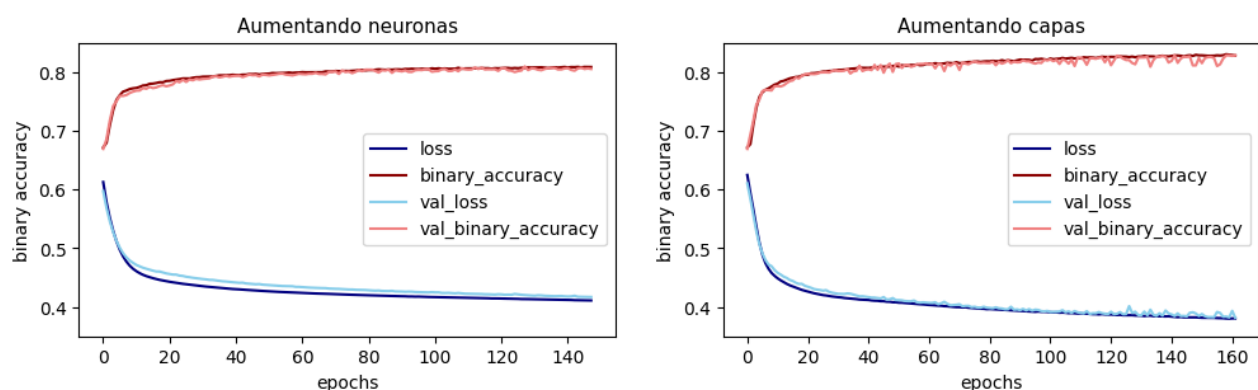


Figura 5: Comportamiento de la red aumentando neuronas y aumentando capas.

Para obtener mayor detalle sobre los resultados, la tabla 9 muestra los resultados numéricos de las ejecuciones. Es significativa la diferencia obtenida aumentando las capas tanto en las métricas como en el sesgo del modelo. El *binary accuracy* ha aumentado un 2.8%, obteniendo un resultado de 0.828 en el conjunto de validación sin aumentar la varianza del modelo. No obstante, el sesgo sigue siendo alto, obteniendo una diferencia de error con respecto al error humano del 12%.

Tabla 9: Métricas obtenidas aumentando neuronas y aumentando capas.

Criterio	Aumentar neuronas	Aumentar capas
Binary accuracy entrenamiento	0.807	0.828
Binary accuracy validación	0.810	0.829
Diferencia binary accuracy entrenamiento y validación	0.25%	0.045%
Diferencia binary accuracy train y error humano	14.28%	12.16%
Coste computacional por época (segundos)	0.725	0.802

A continuación, analizando que el aumento de la complejidad con las capas mejora los resultados, se procede a intentar reducir aún más el sesgo, añadiendo una capa oculta adicional y probando a aumentar las neuronas.

Arquitectura aumentando capas y neuronas

En la tabla 10 se muestran las características e hiperparámetros de las redes diseñadas aumentando el número de capas ocultas y aumentando las neuronas con la misma cantidad de capas.

Tabla 10: Arquitectura red añadiendo una capa más y aumentando neuronas.

Neuronas capas ocultas	Hiperparámetros	Neuronas capas ocultas	Hiperparámetros
	- Función activación: relu - Optimizador: SGD		- Función activación: relu - Optimizador: SGD
64 neuronas	Learning rate: 0.01	128 neuronas	Learning rate: 0.01
64 neuronas	Épocas: 200	128 neuronas	Épocas: 200
64 neuronas	Mini-batch: 128	128 neuronas	Mini-batch: 128
	Early-stopping (patience 20)		Early-stopping (patience 20)

- Análisis del comportamiento

La figura 6 y la tabla 11 no muestran diferencias significativas entre ambos criterios analizados.

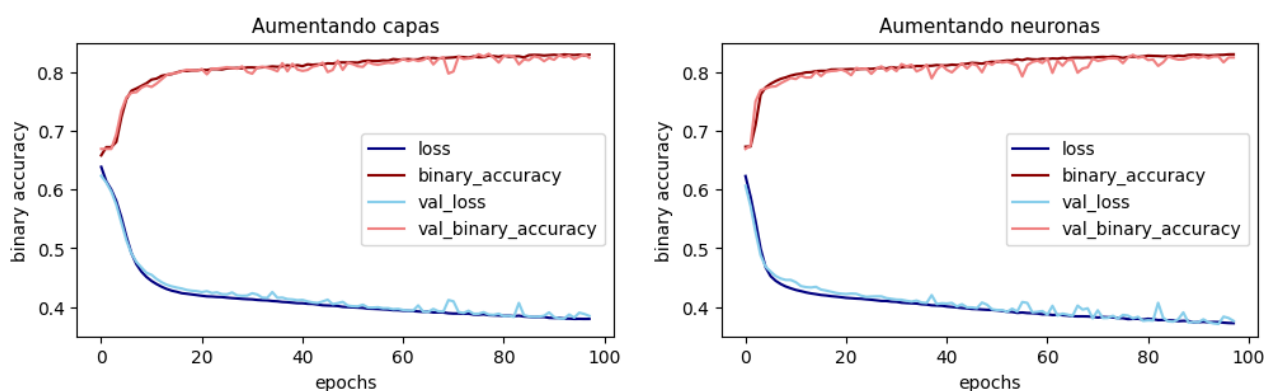


Figura 6: Comportamiento de la red aumentando una capa y aumentando la cantidad de neuronas.

Tabla 11: Métricas obtenidas aumentando neuronas y aumentando capas.

Criterio	Aumentar capas	Aumentar neuronas
Binary accuracy entrenamiento	0.828	0.826
Binary accuracy validación	0.831	0.829
Diferencia binary accuracy entrenamiento y validación	0.324%	0.303%
Diferencia binary accuracy train y error humano	12.19%	12.37%
Coste computacional por época (segundos)	1.030	0.819

Teniendo en cuenta que el sesgo sigue siendo de un 12% aproximadamente, el criterio a seguir a continuación es seguir complejizando el modelo hasta encontrar mejoras significativas.

Arquitectura aumentando capas

Las primeras pruebas mostradas en la tabla 12 consisten en aumentar el número de capas hasta alcanzar 10 capas ocultas todas ellas de 128 neuronas.

Tabla 12: Arquitectura de redes aumentando el número de capas.

Red1	Red2	Red3	Hiperparámetros
5x	7x	10x	• Función activación: relu • Optimizador: SGD • Learning rate: 0.01 • Épocas: 200 • Mini-batch: 128 • Early-stopping (patience 20)
128 neuronas	128 neuronas	128 neuronas	

- Análisis del comportamiento

El criterio llevado a cabo ha resultado en la reducción del sesgo del modelo en consideración, alcanzando un 7% de diferencia con el error humano y alcanzando valores de *binary accuracy* del 0.87 en el conjunto de validación. Estos resultados se pueden observar en la tabla 13 y, además, se puede observar el comportamiento de los entrenamientos a lo largo de las épocas en la figura 7. Cabe destacar que el diseño de la red con 10 capas alcanza el resultado en un número inferior de épocas a diferencia de la red de 7 épocas, el entrenamiento se queda estancado.

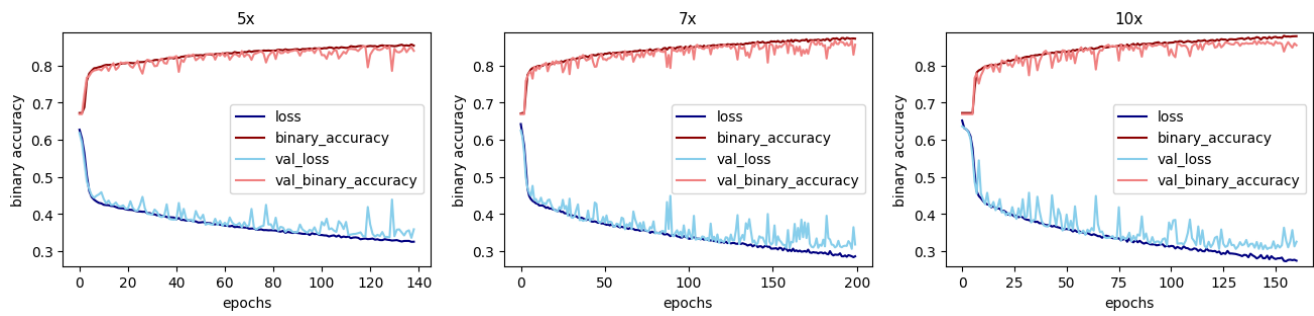


Figura 7: Comportamiento de las redes aumentando las capas.

Tabla 13: Métricas obtenidas aumentando el número de capas

Criterio	5x	7x	10x
Binary accuracy entrenamiento	0.851	0.873	0.873
Binary accuracy validación	0.852	0.869	0.867
Diferencia binary accuracy entrenamiento y validación	0.128%	0.396%	0.665%
Diferencia binary accuracy train y error humano	9.93%	7.727%	7.652%
Coste computacional por época (segundos)	1.004	1.001	1.513

Concluyendo que aumentar las capas reduce en consideración el sesgo sin reducir la varianza, se va a aumentar aún más la complejidad, comparando con los criterios de aumentar capas o mantener las capas aumentando las neuronas.

Arquitectura aumentando capas o neuronas

Tal y como se muestra en la tabla 14, por un lado, se han duplicado las neuronas, y por otro se han añadido tres capas más manteniendo la cantidad de neuronas a 128.

Tabla 14: Arquitectura red aumentando neuronas y aumentando capas.

Neuronas capas ocultas	Hiperparámetros	Neuronas capas ocultas	Hiperparámetros
	• Función activación: relu • Optimizador: SGD • Learning rate: 0.01 • Épocas: 200 • Mini-batch: 128 • Early-stopping (patience 20)		• Función activación: relu • Optimizador: SGD • Learning rate: 0.01 • Épocas: 200 • Mini-batch: 128 • Early-stopping (patience 20)
10x 256 neuronas		13x 128 neuronas	

- Análisis del comportamiento

La figura 8 muestra la tendencia a mejorar que tienen ambos enfoques. Los resultados numéricos de la tabla 15 también muestran un sesgo que va disminuyendo en comparación con las pruebas anteriores.

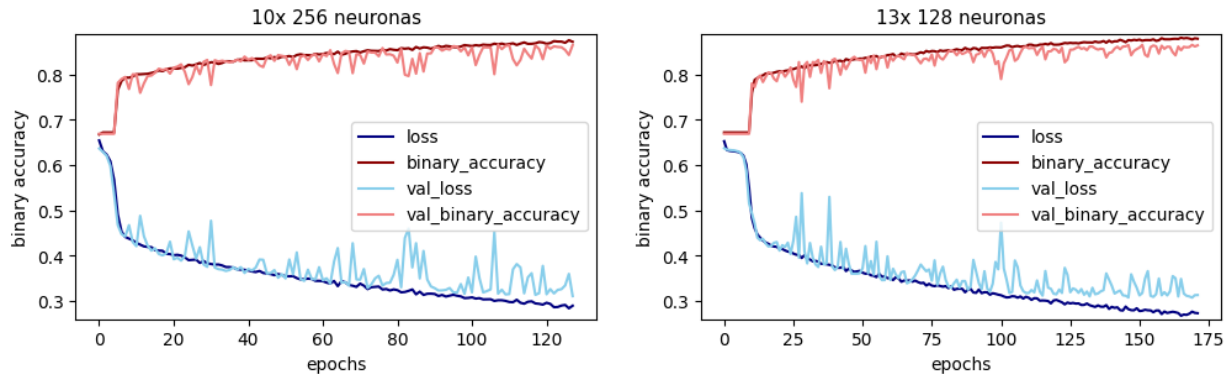


Figura 8: Comportamiento arquitectura aumentando capas o aumentando neuronas.

Tabla 15: Métricas obtenidas aumentando el número de capas y el número de neuronas.

Criterio	10x 256	13x 128
Binary accuracy entrenamiento	0.866	0.875
Binary accuracy validación	0.865	0.865
Diferencia binary accuracy entrenamiento y validación	0.096%	0.917%
Diferencia binary accuracy train y error humano	8.358%	7.538%
Coste computacional por época (segundos)	1.404	1.297

Analizando la mejora constante, se ha procedido a probar con un diseño aumentando en gran escala las capas y las neuronas, con el fin de analizar hasta qué punto el algoritmo sigue mejorando el sesgo sin reducir la varianza en consideración.

Arquitectura aumentando en consideración las capas o las neuronas

El aumento realizado, mostrado en la tabla 16, consiste en mantener 13 capas añadiendo 512 neuronas a cada una de las capas ocultas en el caso del primer diseño, e incrementar hasta 18 el número de capas ocultas con 512 neuronas en cada una de ellas en el segundo diseño.

Tabla 16: Arquitectura de red aumentando en consideración las capas y las neuronas.

Diseño 1	Diseño 2	Hiperparámetros
13x 512 neuronas	18x 512 neuronas	• Función activación: relu • Optimizador: SGD • Learning rate: 0.01 • Épocas: 200 • Mini-batch: 128 • Early-stopping (patience 20)

- Análisis del comportamiento

En la figura 9 se muestra la evolución del entrenamiento durante las épocas. Se percibe que la varianza comienza a aumentar complejizando el problema con ambos enfoques.

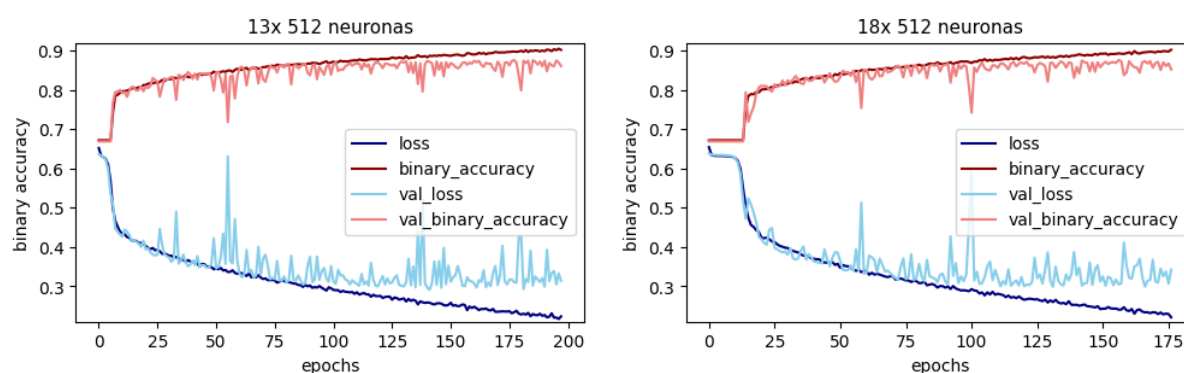


Figura 9: Comportamiento arquitectura aumentando en consideración las capas y las neuronas.

Para obtener un mayor análisis y una mayor comparación con ambos enfoques, se analizan los resultados numéricos de la tabla 17. Se analiza una gran diferencia entre el *binary accuracy* obtenido en el conjunto de entrenamiento y en el conjunto de validación en ambos casos en comparación con las redes diseñadas con anterioridad. Además, aumentando a 18 capas la arquitectura de la red no mejora los resultados a la arquitectura con 13 capas. El *binary accuracy* de entrenamiento disminuye y el de validación se mantiene. Por lo tanto, debido al coste computacional, es mejor solución el diseño de 13 capas ocultas con 512 neuronas cada. Esta última arquitectura mejora el sesgo con respecto a las arquitecturas previas analizadas.

Tabla 17: Métricas obtenidas aumentando el número de capas y el número de neuronas en consideración.

Criterio	13x 512	18x 512
Binary accuracy entrenamiento	0.898	0.893
Binary accuracy validación	0.876	0.876
Diferencia binary accuracy entrenamiento y validación	2.154%	1.696%
Diferencia binary accuracy train y error humano	5.225%	5.656%
Coste computacional por época (segundos)	1.353	1.733

Hasta el momento se ha conseguido reducir el sesgo incrementando la complejidad de la red aumentando neuronas y capas. Teniendo en cuenta que el mejor diseño obtenido hasta el momento corresponde a la arquitectura con 13 capas ocultas y 512 neuronas en cada.

A continuación, se va a modificar la función de activación de la red neuronal reemplazando la función de activación *relu* con otros utilizados en la literatura.

2.5 Arquitectura modificando la función de activación

Las funciones de activación implementadas con el fin de comparar son *relu*, *elu*, *selu*, *softplus* y *leaky relu*. Los restantes parámetros, mostrados en la tabla 18, se han mantenido con el fin de comparar cómo influyen en el resultado de los modelos las distintas funciones de activación.

Tabla 18: Arquitectura red modificando las funciones de activación.

Diseño	Hiperparámetros
13x 512 neuronas	• Función activación: Relu / elu / selu / softplus / leaky relu • Optimizador: SGD • Learning rate: 0.01 • Épocas: 200 • Mini-batch: 128 • Early-stopping (patience 20)

- Análisis del comportamiento

Teniendo en cuenta los resultados obtenidos para cada función de activación implementada, ilustrados en la figura 10 y mostrados en la tabla 19, se ha concluido que *relu*, *selu* y *leaky relu* son las funciones de activación que mejor comportamiento tienen con el optimizador SGD. La función *elu* presenta un entrenamiento menor de los datos, estancándose en las primeras iteraciones sin alcanzar mejora alguna. La función de activación *selu* alcanza un sesgo muy bajo con respecto al error humano del 4%, no obstante, la varianza aumenta, calculando una diferencia entre el conjunto de entrenamiento y de validación del 4%. En el caso del optimizador se ha optado por la arquitectura con la función de activación *relu* por ser el diseño con menor sesgo y menor varianza.

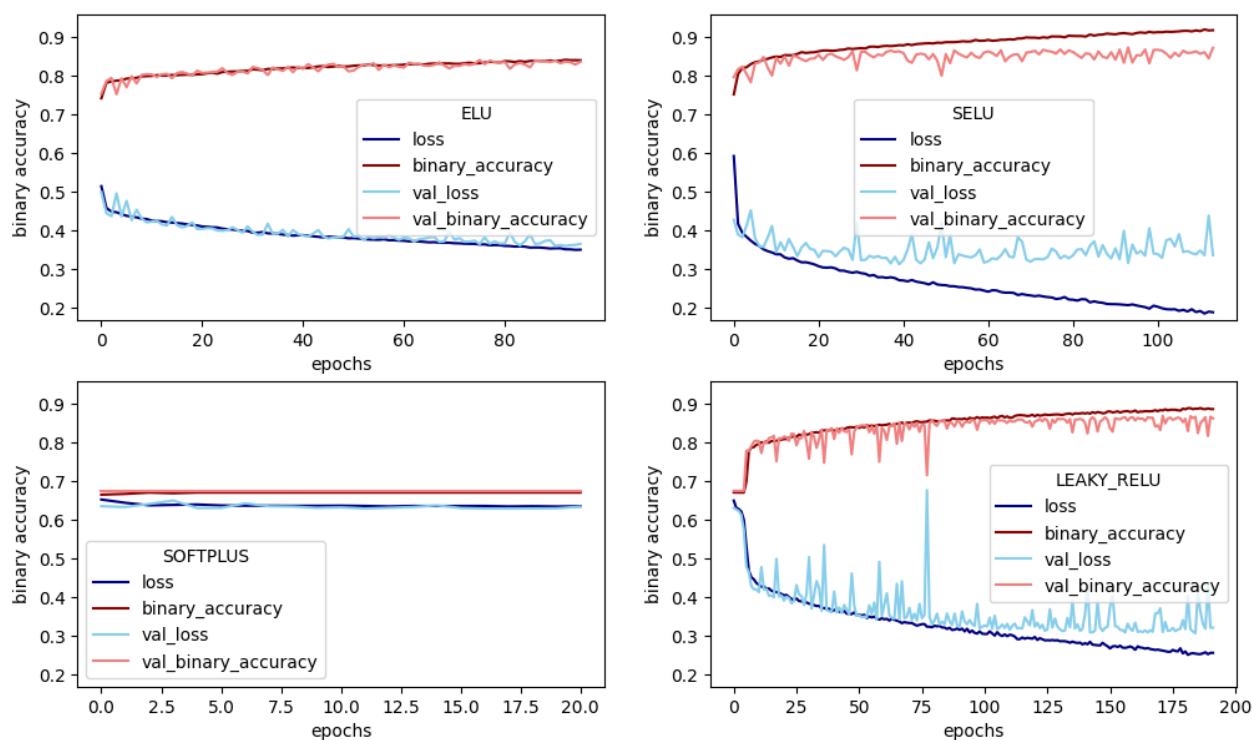


Figura 10: Comportamiento redes con diferentes funciones de activación.

Tabla 19: Métricas de los resultados para las redes con distintas funciones de activación.

Criterio	elu	selu	softplus	Leaky relu
Binary accuracy entrenamiento	0.833	0.910	0.666	0.884
Binary accuracy validación	0.841	0.874	0.676	0.870
Diferencia binary accuracy entrenamiento y validación	0.837%	4.047%	0.982%	1.392%
Diferencia binary accuracy train y error humano	11.691%	3.581%	28.406%	6.621%
Coste computacional por época (segundos)	2.251	1.452	2.509	1.842

En las secciones posteriores se procede a implementar modelos cambiando los optimizadores con las funciones de activación relu, leaky relu y selu debido a sus buenos resultados con el optimizador SGD. El objetivo consiste en analizar el comportamiento de otros optimizadores analizados en la literatura tal y como se ha procedido con el optimizar SGD.

Además, al cambiar de optimizador, se ha decidido modificar la arquitectura de la red buscando la arquitectura que mejor comportamiento muestre con el optimizador con el que se optimizan los parámetros de la red neuronal, con el objetivo de reducir el sesgo de los modelos para cada optimizador, siguiendo los pasos anteriormente analizados con el SGD y analizando cada ejecución.

2.6 Arquitectura modificando el optimizador

Para cada optimizador se ha modificado la estructura 1) aumentando el número de capas, 2) incrementando/disminuyendo en cada capa el número de neuronas, 3) modificando el *batch size*, 4) modificando el *learning rate* y 5) aplicando decrecimiento del *learning rate*. Los optimizadores son sensibles al *learning rate* establecido, ya que experimentalmente se ha analizado que una misma tasa de aprendizaje no funciona igual para todos los optimizadoras, impidiendo el entrenamiento en algunos casos. Por ende, además de modificar el número de capas y neuronas se ha reparado exhaustivamente en la selección del *learning rate*. Cabe mencionar que el decrecimiento de la tasa de aprendizaje implementado para diferentes optimizadores no ha resultado en un entrenamiento estable, aumentando en consideración el sobreajuste de los datos. Siempre hemos probado utilizando un *learning rate* inicial 10 veces el inicial habitual y decaerlo cada época 0.98 exponencialmente.

En la tabla 20 se muestran los diseños óptimos encontrados para cada optimizador comparando y reparando en el comportamiento de múltiples pruebas.

Tabla 20: Arquitectura de red de los mejores resultados obtenidos para diferentes optimizadores.

Optimizador	SDG	Adam	Rmsprop	Momentum	Nesterov
Capas (neuronas)	13x (512)	(4096 2048 1024 512 256 128 64)	(4096 4096 4096 1024 256 64)	(4096 2048 1024 512 256 64)	(1024 1024 512 512 512 256 64)
Función activación	Relu	Leaky-relu	Selu	Relu	Selu
Learning rate	0.01	0.0001	0.0001	0.01	0.01
Épocas	200	200	200	200	200
Mini-batch	128	128	128	128	128
Early stopping (patience)	20	20	20	20	20

- Análisis del comportamiento

Analizando el comportamiento en la figura 11 y los resultados en la tabla 21, se concluye que los optimizadores *Adam* y *Momentum* permiten reducir más el sesgo que los restantes modelos. No obstante, se observa una gran varianza entre los resultados del conjunto de varianza y del conjunto de validación.

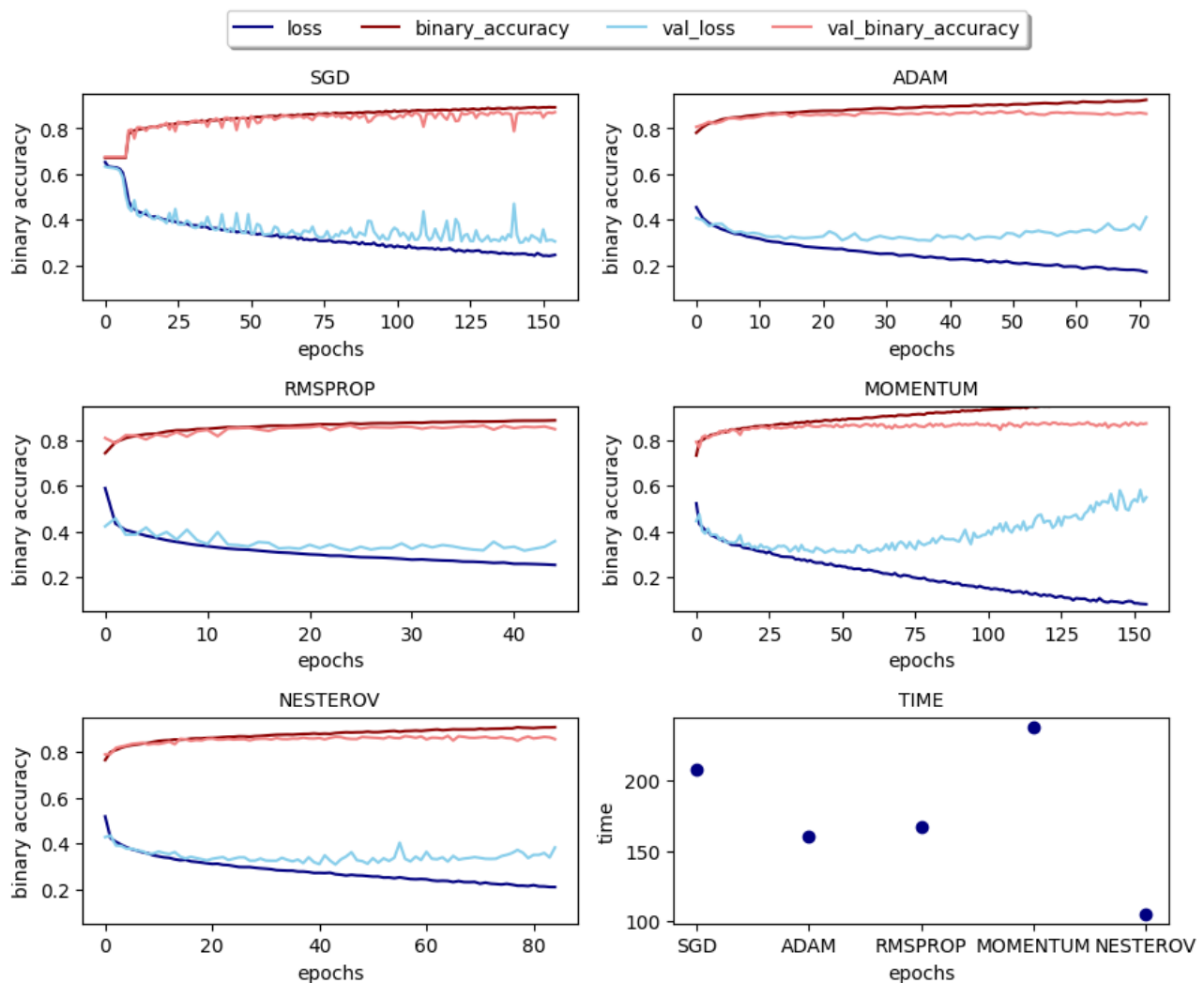


Figura 11: Comportamiento redes de los mejores resultados obtenidos para diferentes optimizadores.

Tabla 21: Métricas de los mejores resultados obtenidos para diferentes optimizadores.

Criterio	SDG	Adam	RMSprop	Momentum	Nesterov
Binary accuracy entrenamiento	0.898	0.906	0.873	0.957	0.897
Binary accuracy validación	0.876	0.877	0.864	0.879	0.870
Diferencia binary accuracy entrenamiento y validación	2.154%	2.940%	0.931%	7.875%	2.616%
Diferencia binary accuracy train y error humano	5.225%	4.384%	7.662 %	-0.744%	5.343%
Coste computacional por época (segundos)	1.353	2.227	3.707	1.538	1.236

A partir de estas soluciones, se seleccionan los modelos *momentum* y *Adam* para reducir la varianza. Se realizan pruebas generales con el de *momentum* y tras analizar los resultados se aplican las capas y técnicas que mayor rendimiento han generado al modelo de Adam con el fin de reducir la varianza en ambos.

2.7 Reducción varianza en modelo Momentum

Las técnicas implementadas para reducir la varianza para el mejor modelo obtenido hasta la actualidad con el optimizador de *Momentum* son: *batch-normalization* antes de la función de activación, *batch-normalization* después de la función de activación, inicializador de pesos *He* y *Xavier*, *dropout*, regularizaciones L1 y L2 y la combinación de *batch normalization* con *dropout*.

- Análisis del comportamiento

Observando el comportamiento de los modelos aplicando las técnicas mencionadas en la figura 12, se concluye que las regularizaciones L1 y L2 no han ayudado a mejorar el comportamiento del modelo. Asimismo, se analiza que la técnica *batch normalization* mejora cuando se aplica antes de la función de activación. Los resultados numéricos se pueden observar en la tabla 22.

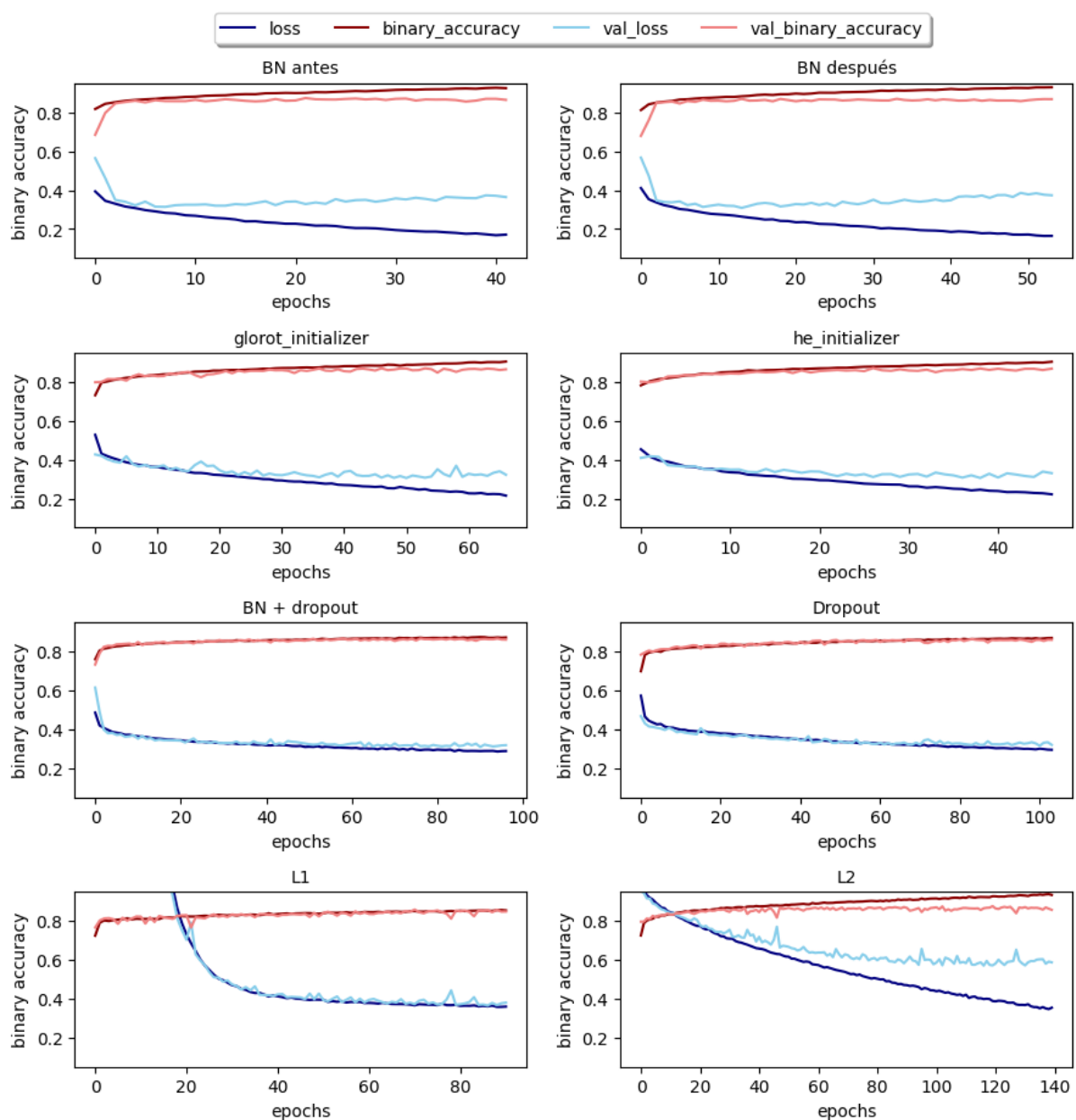


Figura 12: Comportamiento de una red con mecanismos para reducir la varianza.

Tabla 22: Métricas de una red con mecanismos para reducir la varianza.

Criterio	BN antes	BN después	Xavier	He
Binary accuracy entrenamiento	0.900	0.913	0.884	0.879
Binary accuracy validación	0.874	0.870	0.870	0.871
Diferencia binary accuracy entrenamiento y validación	2.643%	4.291%	1.341%	0.751%
Diferencia binary accuracy train y error humano	4.957%	3.114 %	6.618%	7.124%
Coste computacional por época (segundos)	3.255	3.114	2.066	1.844

Criterio	Dropout(0.3)	BN + Dropout(0.3)	L1	L2
Binary accuracy entrenamiento	0.864	0.871	0.851	0.927
Binary accuracy validación	0.864	0.867	0.859	0.876
Diferencia binary accuracy entrenamiento y validación	0.014%	0.414%	0.803%	5.070%
Diferencia binary accuracy train y error humano	8.634%	7.931%	9.919 %	2.282 %
Coste computacional por época (segundos)	2.123	2.640	2.594	2.248

En general, todas las técnicas han ayudado a sobreajustar menos, pero ninguna ha resultado en un aumento del *accuracy* de validación. Parece prometedor utilizar normalización *batch* y *dropout* juntos para entrenar modelos más grandes, ya que se observa un rendimiento aceptable y muy poco sobreajustado, cosa que no ocurre en las técnicas de inicialización que sobre ajustan más. No obstante, reparando los resultados, el *dropout* con un parámetro de 0.3 ha hecho que el modelo no tenga opción de aprender más.

Las siguientes pruebas van a variar el parámetro del *dropout* para intentar reducir la varianza recuperando el bajo sesgo. Asimismo, como las técnicas del *batch normalization* y el *dropout* han ayudado a reducir la varianza causando una disminución en el sesgo, se implementan diseños de arquitecturas aumentando capas y aplicando otras técnicas como el decrecimiento del *learning rate* a lo largo de las épocas.

2.8 Reducción varianza y reducción del sesgo en el modelo Momentum

Los tres diseños implementados son: 1) *Batch normalization* con *dropout* de parámetro 0.2, 2) *batch normalization* con *dropout* de parámetro 0.15 añadiendo una capa de 4096 y otra de 2048 y 3) *batch normalization* con *dropout* de parámetro 0.15 añadiendo una capa de 4096 y otra de 2048 y reduciendo el *learning rate* exponencialmente. Las pruebas implementadas están visibles en la tabla 23.

Tabla 23: Diseños de redes para disminuir la varianza y aumentar el sesgo.

Diseño general	Hiperparámetros	Diseño 1	Diseño 2	Diseño 3
(4096 2048 1024 512 256 64)	• Función activación: Relu	• BN	• BN	• BN
	• Optimizador: MOMENTUM	• Dropout (0.2)	• Dropout (0.15)	• Dropout (0.15)
	• Learning rate: 0.01		• + 1 capa 4096	• + 1 capa 4096
	• Épocas: 200		• +1 capa 2048	+1 capa 2048
	• Mini-batch: 128			• Learning rate exponential decay
	• Early-stopping (patience 20)			

- Análisis del comportamiento

Los resultados para los 3 diseños de arquitectura de red se analizan en la figura 13 y tabla 24.

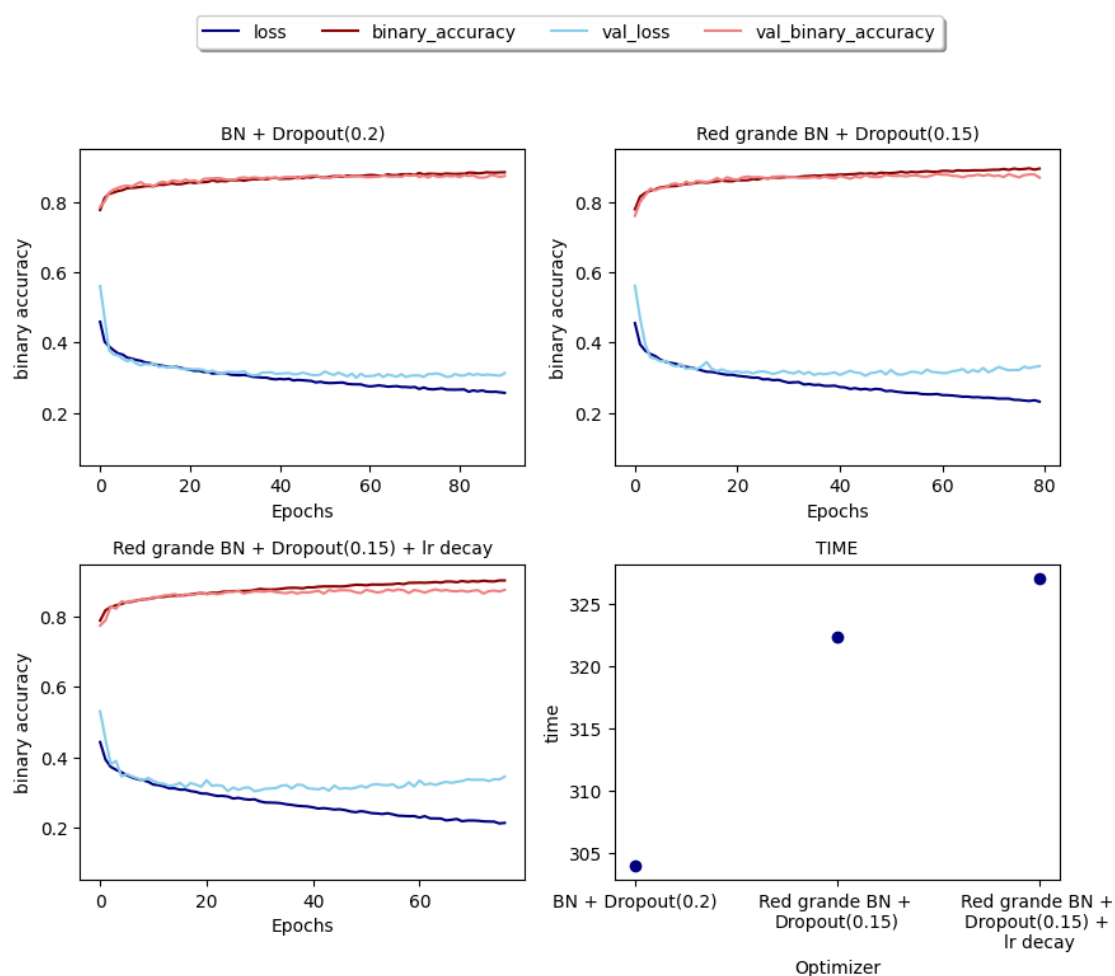


Figura 13: Comportamiento mejor modelo momentum con técnicas para reducir varianza y aumentar sesgo.

Tabla 24: Métricas mejor modelo momentum con técnicas para reducir varianza y aumentar sesgo.

Criterio	Diseño 1	Diseño 2	Diseño 3
Binary accuracy entrenamiento	0.877	0.886	0.895
Binary accuracy validación	0.877	0.880	0.878
Diferencia binary accuracy entrenamiento y validación	0.059%	0.057 %	1.678 %
Diferencia binary accuracy train y error humano	7.226 %	6.421%	5.536 %
Coste computacional por época (segundos)	3.340	4.030	4.247

En efecto, el *dropout* con valores más bajos resulta en un mejor entrenamiento, ya que permite al modelo aprender más. Se ha obtenido el mejor modelo hasta ahora analizando el *binary accuracy* en el conjunto de validación (0.88 en validación) con el diseño 2, es decir aumentando dos capas de 4096 y 2048 y añadiendo *batch normalization* y *dropout* de 0.15. Utilizar el *learning rate decay* ha hecho que sobreaprenda más y obtenga métricas inferiores al diseño 2.

A continuación, partiendo de los resultados obtenidos con el optimizador de *momentum* y concluyendo que el *dropout* y el *batch normalization* ayudan a reducir la varianza del modelo, se comparan ambos métodos con el mejor modelo obtenido con el optimizador de *Adam*.

2.9 Reducción varianza en modelo Adam

En un primer paso, se han implementado por un lado el *batch normalization*, y por otro el *batch normalization* junto con un *dropout* con parámetro 0.3 para analizar el impacto que tienen con el optimizador Adam. En la tabla 25 se muestran los resultados obtenidos para ambas técnicas.

Tabla 25: Métricas mejor modelo Adam con técnicas para reducir la varianza del modelo.

Criterio	BN antes	BN + Dropout(0.3)
Binary accuracy entrenamiento	0.919	0.866
Binary accuracy validación	0.877	0.867
Diferencia binary accuracy entrenamiento y validación	4.229%	0.186%
Diferencia binary accuracy train y error humano	3.096%	8.448%
Coste computacional por época (segundos)	3.586	3.473

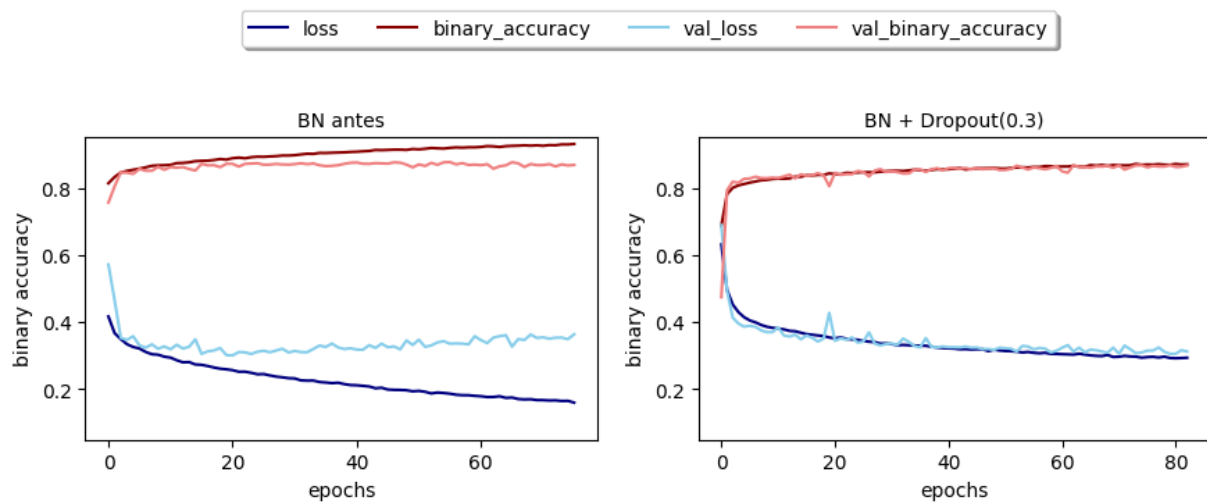


Figura 14: Comportamiento de las técnicas de reducción de varianza en Adam

Se aprecia un fenómeno similar a lo visto con *momentum*, aunque el *dropout* ayuda reducir la varianza el sesgo disminuye. Por lo tanto, a continuación, se diseñan arquitecturas para ayudar tanto a reducir la varianza como reducir el sesgo.

2.10 Reducción varianza y sesgo en modelo Adam

Se implementan 4 diseños distintos para la arquitectura de la red. En los 4 casos se aumentan las capas del modelo (una capa adicional con 4096 neuronas y otra de 2048 neuronas). En el primer diseño se establece un *dropout* con parámetro 0.3, a diferencia del segundo diseño con un parámetro de 0.2. El tercer diseño establece un parámetro 0.2 para el *dropout* añadiendo un decrecimiento exponencial del *learning rate* a lo largo de las épocas y el diseño 4 se diferencia del anterior modificando el parámetro del *learning rate* a 0.15. Las características de los 4 diseños están visibles en la tabla 26.

Tabla 26: Diseños de redes para disminuir la varianza y el sesgo en modelo Adam.

Diseño general	Hiperparámetros	Diseño 1	Diseño 2	Diseño 3	Diseño 4
(4096 2048 1024 512 256 128 64)		• BN	• BN	• BN	• BN
	• Función activación: Leaky-relu	• Dropout (0.3)	• Dropout (0.2)	• Dropout (0.2)	• Dropout (0.15)
	• Optimizador: ADAM	• + 1 capa 4096	• + 1 capa 4096	• + 1 capa 4096	• + 1 capa 4096
	• Épocas: 200	+1 capa	+1 capa	• +1 capa	+1 capa
	• Mini-batch: 128	2048	2048	2048	2048
	• Early-stopping (patience 20)	• Lr fijo: 0.0001	• Lr fijo: 0.0001	• Lr exp decay	• Lr exp decay

- Análisis del comportamiento

Los resultados mostrados por la figura 15 y la tabla 27 permiten concluir que el diseño 3 tiene el sesgo más bajo en comparación con los restantes y que el diseño 4 tiene una varianza significativamente baja, del 0.4%, comparando los resultados del conjunto de entrenamiento y del conjunto de validación.

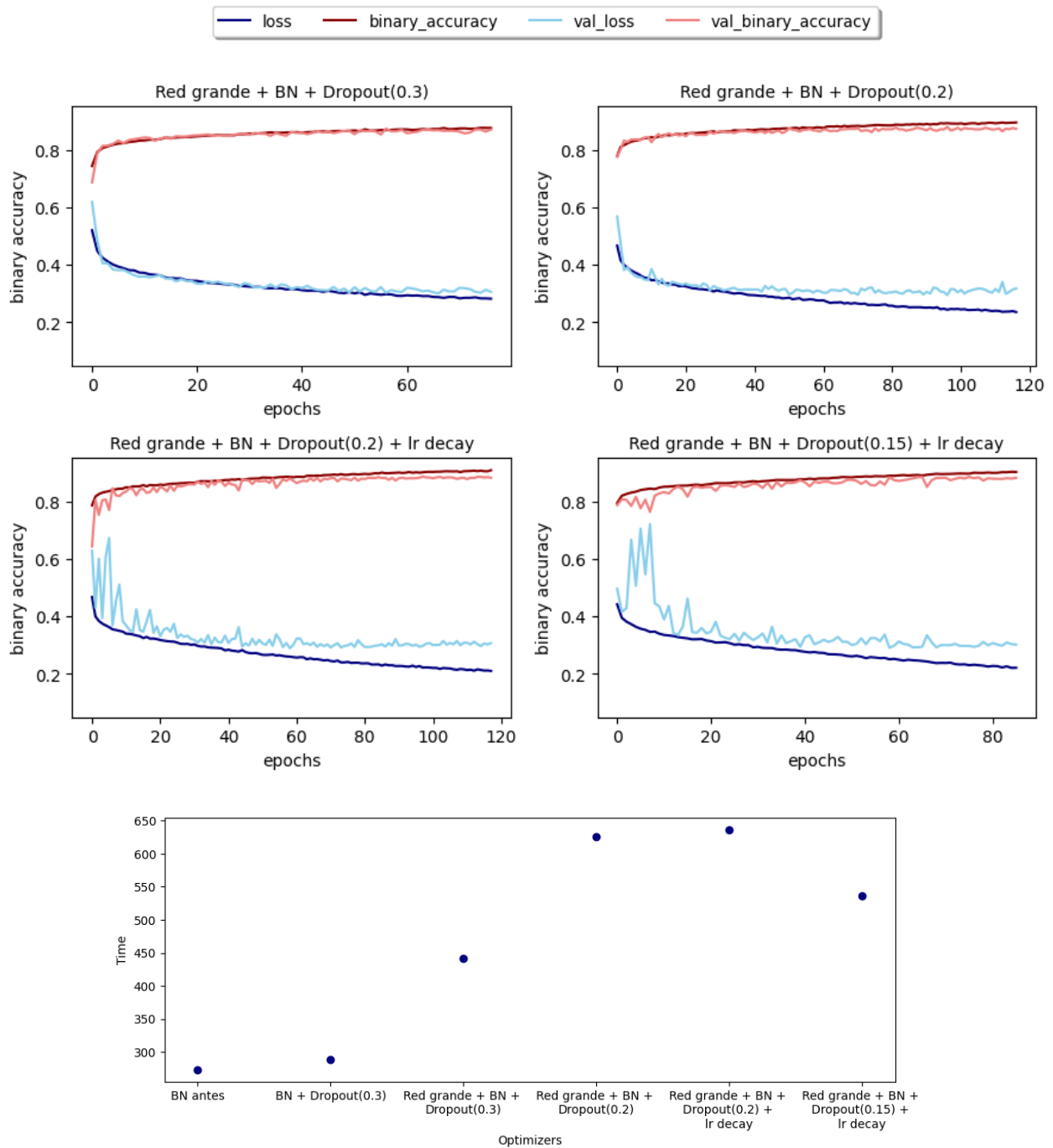


Figura 15: Comportamiento de los modelos Adam para reducir sesgo y varianza

Tabla 27: Métricas mejor modelo Adam con técnicas para reducir la varianza y sesgo del modelo

Criterio	Diseño 1	Diseño 2	Diseño 3	Diseño 4
Binary accuracy entrenamiento	0.868	0.892	0.901	0.889
Binary accuracy validación	0.873	0.879	0.886	0.885
Diferencia binary accuracy entrenamiento y validación	-0.493%	1.330 %	1.468 %	0.469 %
Diferencia binary accuracy train y error humano	8.175%	5.773 %	4.919 %	6.056%
Coste computacional por época (segundos)	5.742	5.346	5.388	6.234

Los mejores resultados son aumentando más la red y utilizando un valor de *dropout* más pequeño y *learning rate decay* exponencial. Este se parametriza con *learning rate* inicial 0.001 (10 veces más alto que en el modelo base), decreciendo con base 0.98 cada época. Teniendo en cuenta que el diseño 4, además de ser el que menor varianza muestra, tiene un sesgo bajo; por lo que se considera hasta el momento el modelo con mejor comportamiento mostrando un balance entre el sesgo y la varianza. El tiempo de entrenamiento es superior a los restantes, pero se trata de un coste computacional aceptable y permite un comportamiento mejor en el balance entre el sesgo y de la varianza en comparación a lo anterior analizado.

Asimismo, comparando la mejor arquitectura obtenida con el optimizador *Adam* y el con optimizador *Momentum*, se ha optado por el *Adam* ya que manteniendo una misma varianza ambos, la arquitectura con *Adam* alcanza un sesgo inferior.

2.11 Últimos modelos para disminución de sesgo y varianza Adam

A continuación, se intenta aumentar el tamaño de la red con el diseño 4 mencionado con el optimizador *Adam*. También se determinan diferentes probabilidades de *dropout* para lograr un aumento en el *binary accuracy* sin sobreaprender ni regularizar demasiado.

En concreto, partiendo del mejor modelo encontrado en la sección de reducción de sesgo y varianza de Adam, se aplica un proceso iterativo donde surgen cuatro diseños adicionales, los mostrados en la tabla 28.

Tabla 28: Métricas diferentes arquitecturas para reducir sesgo y varianza con el optimizador Adam.

Diseño general	Hiperparámetros	Diseño 1	Diseño 2	Diseño 3	Diseño 4
(4096 4096 2048 2048 1024 512 256 128 64)	• Función activación: Leaky-relu • Optimizador: ADAM • Learning rate decay • Épocas: 200 • Mini-batch: 128 • BN • Paciencia 30	• Dropout (0.15) • + 1 capa 4096 +1 capa 2048	• Dropout (0.2) • + 1 capa 4096 +1 capa 2048	• BN • Dropout (0.2) • + 1 capa 4096 • +1 capa 2048 2048 + 1 capa 1024 + 1 capa 512	• Dropout (0.25) • + 1 capa 4096 +1 capa 2048

- El primero solo aumenta la complejidad base añadiendo dos capas adicionales. Los resultados obtenidos muestran un sobreajuste mayor de lo esperado y con el objetivo de reducirlo, surge el siguiente modelo.

- El segundo, mantiene la complejidad aumentada del primero, pero, con el objetivo de reducir la varianza y el sobreajuste general del modelo, se aumenta la probabilidad de *dropout* a 0.2. Los resultados muestran una disminución del sobreajuste pero sin signos de pérdida en el *binary accuracy* de validación, algo muy positivo y que nos motivó a profundizar en esta configuración.

- El tercero es una prueba adicional en busca de mejorar el resultado anterior. Para ello se incrementa ligeramente la complejidad del segundo modelo añadiendo dos capas, una de 1024 y otra de 512. El resultado muestra la parada anticipada del proceso de entrenamiento, incluso antes que los modelos 1 y 2. Por esto, los resultados obtenidos muestran un modelo con un poco menos de *binary accuracy*, tanto en entrenamiento como en validación, respecto al segundo modelo.

Habiendo considerado estos tres modelos, se selecciona el segundo como mejor modelo debido a su baja varianza. Habiendo visto con el tercer modelo que este segundo modelo no iba a mejorar si se aumenta más la complejidad, se decide plantear un último modelo con el objetivo de reducir aún más el sobreajuste, con la esperanza de que muestre ese comportamiento previo con el que se mantenía el *accuracy* de validación mientras se disminuía el de entrenamiento.

- Ese cuarto y último modelo se caracteriza por mantener las opciones de diseño del segundo modelo, pero con un valor mayor de *dropout* (0.25). El planteamiento al realizar esto es que, si el modelo mantiene el valor de *accuracy* en validación mientras se reduce en entrenamiento, el modelo estará generalizando mejor, y por tanto tendrá mejor rendimiento ante un conjunto de datos nunca visto como es el de test final. Los resultados de este modelo son los esperados, un menor valor de entrenamiento mientras se mantiene el valor de validación; y dado que ambos valores de *binary accuracy* son tan cercanos (0.015 de diferencia), se ha optado por escogerlo como nuestro modelo final. Por un lado, es un modelo que reduce la varianza evitando el sobreajuste y permitiendo la generalización, y, por otro lado, tiene un sesgo bajo con respecto al error humano considerado.

Los resultados de los cuatro modelos mencionados se pueden analizar tanto visualmente en la figura 16 como numéricamente en la tabla 29.

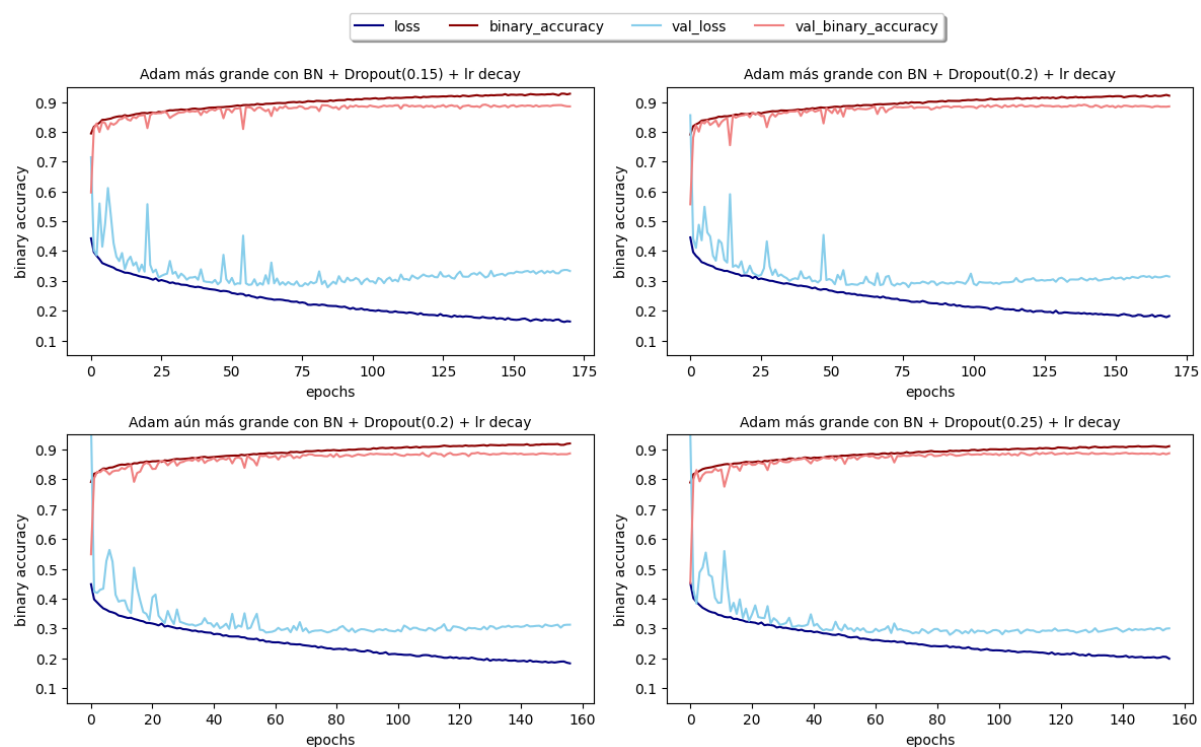


Figura 16: Comportamiento diferentes arquitecturas para reducir sesgo y varianza con el optimizador Adam

Tabla 29: Métricas de los últimos modelos propuestos para reducir sesgo y varianza

Criterio	Diseño 1	Diseño 2	Diseño 3	Diseño 4
Binary accuracy entrenamiento	0.922	0.915	0.913	0.906
Binary accuracy validación	0.891	0.891	0.890	0.891
Diferencia binary accuracy entrenamiento y validación	3.116 %	2.381 %	2.330 %	1.516 %
Diferencia binary accuracy train y error humano	2.775 %	3.537 %	3.699 %	4.429%
Coste computacional por época (segundos)	7.112	6.724	7.067	7.933

3 Resultados finales

El mejor modelo dada la información de sesgo y varianza es el entrenado con Adam, con estructura de capas (4096, 4096, 4096, 2048, 2048, 2048, 1024, 512, 256, 128, 64, 1), con *learning rate decay exponencial* con *learning rate* inicial en 0.001 (10 veces más alto que en el modelo base), decreciendo con base 0.98 cada época, tamaño de *mini-batch* de 128, *dropout* con probabilidad 0.25, *normalización batch*, *early stopping* de paciencia 30 y función de activación *leaky relu*. Como se muestra en la figura 17, el inicio del entrenamiento es ruidoso por el hecho de tener un *learning rate* alto, pero más adelante se vuelve mucho más estable.

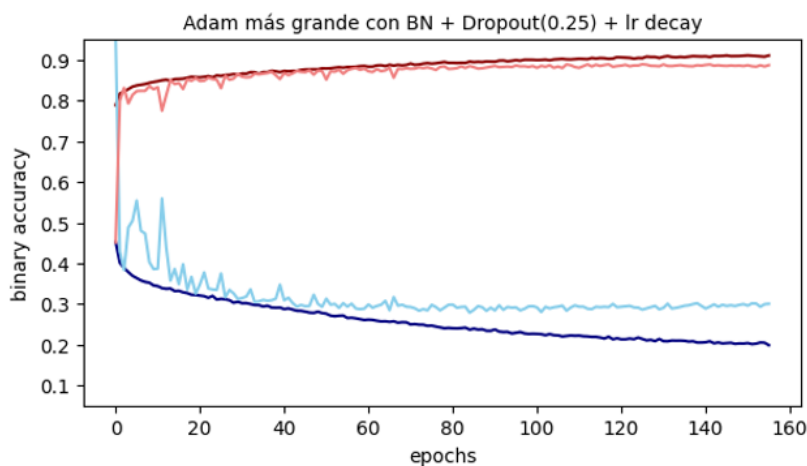


Figura 17: Comportamiento modelo final

Se guarda el mejor modelo obtenido reparando en la métrica binary accuracy en el conjunto de evaluación asegurando una varianza baja y un sesgo bajo como resultado final. En este punto se recupera el conjunto de testeo que contiene el 10% del conjunto de datos y se predice con el modelo entrenado, obteniendo un binary accuracy de 0.8646. La tabla 30 muestra el resumen de todas las métricas analizadas del modelo.

Tabla 30: Métricas modelo final.

Criterio	Diseño 4
Binary accuracy entrenamiento	0.906
Binary accuracy validación	0.891
Binary accuracy test final	0.8646
Diferencia binary accuracy entrenamiento y validación	1.516 %
Diferencia binary accuracy train y error humano	4.429%
Coste computacional por época (segundos)	7.933

Pese a ser un modelo con sesgo y varianza bastante bajos, los resultados de test final son un 2.5% inferiores a lo que ha dado en validación, lo que es razonable por la poca cantidad de datos de los que

disponemos, lo que puede llevar a *splits* no perfectamente homogéneos. Esto podría mejorarse aplicando CV sobre los datos (lo cuál sería muy costoso) o con *data augmentation*.

En la introducción se realizó una reflexión sobre las posibles implicaciones de tener errores relativos a la matriz de confusión y en la figura 18 se conoce qué errores hay con los datos obtenidos. Se recalca que no se va a valorar cuál de estos es más dañino por falta de información de la estrategia del hotel.

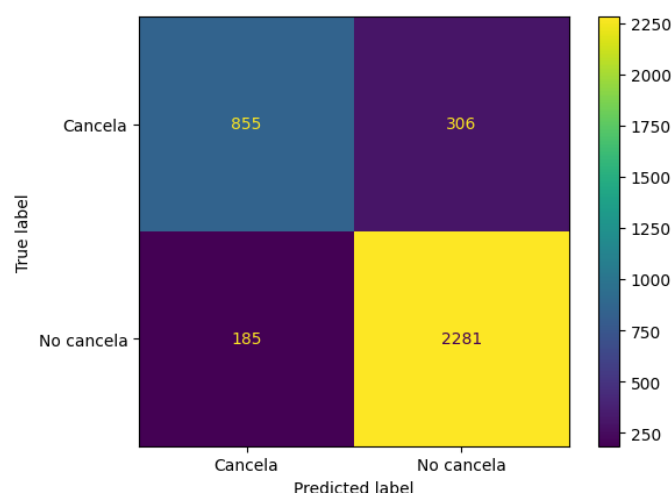


Figura 18: Matriz de confusión conjunto de testeo.

Sobre la diagonal principal están los aciertos, y sobre la otra diagonal los errores:

Se calcula la precisión y el *recall* de ambas clases:

Tabla 31: Precision y recall obtenidos en el conjunto de testeo.

	Precisión	Recall
Clase no cancela	0.8817	0.9249
Clase cancela	0.8221	0.7364

Se aprecian mejores resultados en ‘no cancela’, la clase mayoritaria, lo que es razonable porque el modelo ha visto más datos de esa clase y ha podido aprender mejor sobre ella.

- La precisión de la clase “no cancela” indica la proporción del modelo que, habiendo predicho que el cliente no cancelaba, en la realidad no ha cancelado su reserva. Es decir, el modelo ha predicho correctamente a 2281 clientes que realmente no cancelaron, al 88%. Ha habido 306 clientes que siendo “cancela” han sido clasificados incorrectamente como “no cancela”. La precisión es bastante alta, por lo que el hotel puede confiar bastante en la predicción que clasifica como que no se cancela.
- El *recall* de esta clase señala la proporción de clientes que no han cancelado y que el modelo ha predicho correctamente que no cancelaban. Es decir, ha confundido a 185 clientes que no iban a cancelar como cancelaciones. El *recall* es el dato más alto, el hotel puede estar muy

seguro de que no se van a predecir como cancelaciones los casos que realmente son no cancelaciones.

En cuanto a la clase “cancela” las medidas *precision* y *recall* muestran la información inversa.

- La precisión sería la proporción del modelo que, habiendo predicho que el cliente iba a cancelar, realmente sí que ha cancelado. Es decir, a 185 clientes que había predicho que iban a cancelar, realmente no han cancelado. La precisión es relativamente alta, por lo que el hotel puede estar bastante seguro de que, si la predicción clasifica como que se cancela, lo más segura es que se va a cancelar la reserva.
- El *recall* muestra la proporción de reservas que han sido canceladas y que el modelo ha predicho correctamente que se iba a cancelar. Es decir, ha confundido a 306 clientes que iban a cancelar como no cancelaciones. Este es el dato más bajo, el hotel no puede estar seguro de que, si el cliente va a cancelar, el modelo lo prediga correctamente.

4 Conclusiones

Se ha resuelto el problema con un *accuracy* aceptable (86.46%) tras realizar muchas pruebas con muchas técnicas de reducción de sesgo y varianza. Para llegar a ello se han ido intercambiando los objetivos de reducir sesgo y varianza iterativamente hasta llegar a un punto donde ambos estaban equilibrados. Muchas veces ocurrió que el sesgo bajó mucho y luego al pasar a reducir la varianza el sesgo subió de nuevo por la regularización aplicada. En cada fase se fue seleccionando el modelo que mejor lograba el objetivo teniendo en cuenta que balanceara bien el otro.

Otra conclusión es que en el proceso de ir eligiendo entre varias opciones ha ocurrido por ejemplo que no había ninguna función de activación superior a las demás, sino que iba variando en función de la elección previa de otros hiperparámetros como la estructura de la red o el optimizador. Es muy probable que exista una red de calidad similar a nuestra red final que utilice otra combinación que nosotros no hemos descubierto o explorado en profundidad.

Gracias a la matriz de confusión se sabe que se puede confiar más en acertar la clase de ‘no cancelar’ que la otra. El hecho de tener clases desbalanceadas es muy negativo porque la clase minoritaria no se aprende bien y además los *splits* de los datos en entrenamiento, validación y test tienden a estar más sesgados, lo que reduce la fiabilidad del modelo cuando se ponga en producción.

Se propone como línea futura generar datos de la clase minoritaria con alguna técnica de sobremuestreo.

Además, si se dispusiera de información sobre la estrategia que quiere seguir el hotel a partir de las predicciones del modelo podría cambiarse la función de pérdida para reducir el riesgo de error de falsos positivos o negativos para reducir la probabilidad de causar perjuicios graves. Por ejemplo, si el hotel va a mantener libres habitaciones reservadas que la red dice que se van a cancelar, habría que reducir el riesgo de predecir una falsa cancelación.